

CSA6501
Generative AI
Lab Programs
(01 – 20)

Y. Jayesh Ranjan
192472201

1. Aim:

To install Matplotlib and display a simple line graph.

Algorithm

1. Import the Matplotlib library.
2. Create X and Y values.
3. Plot the line graph.
4. Add title and labels.
5. Display the graph.

Code:

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
plt.plot(x, y)
```

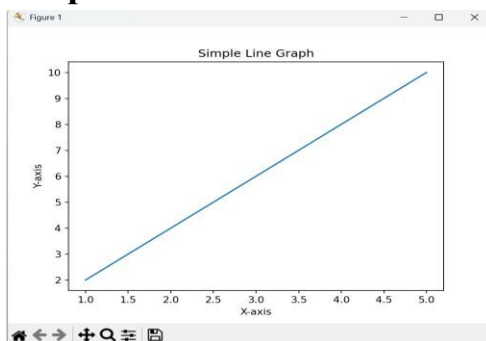
```
plt.title("Simple Line Graph")
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.show()
```

Output:



Result:

Thus, a simple line graph was created successfully using Matplotlib.

2. Aim:

To verify the installed version of Scikit-learn.

Algorithm

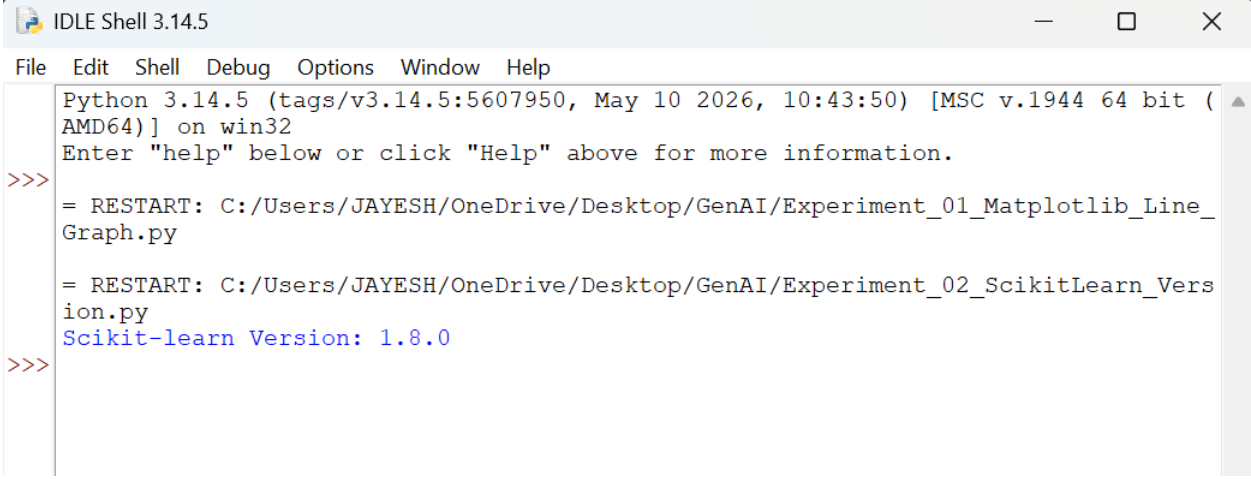
1. Import Scikit-learn.
2. Print the version.
3. Display the output.

Code:

```
import sklearn
```

```
print("Scikit-learn Version:", sklearn.__version__)
```

Output:



```
IDLE Shell 3.14.5
File Edit Shell Debug Options Window Help
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/JAYESH/OneDrive/Desktop/GenAI/Experiment_01_Matplotlib_Line_Graph.py
= RESTART: C:/Users/JAYESH/OneDrive/Desktop/GenAI/Experiment_02_ScikitLearn_Version.py
Scikit-learn Version: 1.8.0
>>>
```

Result:

Thus, the installed version of Scikit-learn was displayed successfully.

3. Aim

To create a NumPy array containing the first 20 natural numbers.

Algorithm

1. Import NumPy.
2. Create an array using `arange()`.
3. Display the array.

Code

```
import numpy as np
```

```
arr = np.arange(1, 21)
```

```
print(arr)
```

Output:

```
>>> = RESTART: C:/Users/JAYESH/OneDrive/Desktop/GenAI/Experiment_03_NumPy_Array.py =  
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]  
>>> |
```

Result:

Thus, the NumPy array containing the first 20 natural numbers was created successfully.

4. Aim

To create a Pandas DataFrame containing student names and grades.

Algorithm

1. Import Pandas.
2. Create a dictionary.
3. Convert it into a DataFrame.
4. Display the DataFrame.

Code

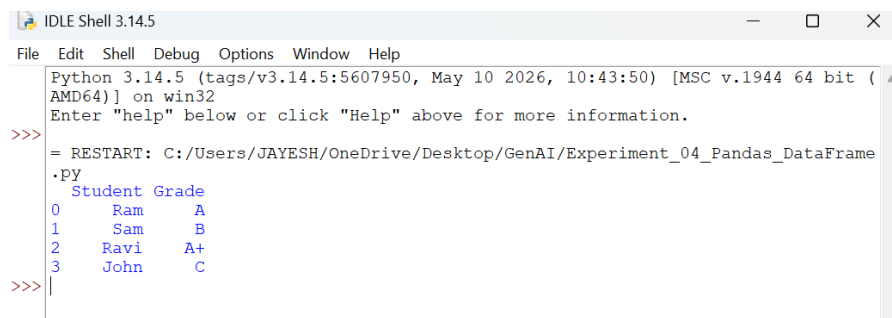
```
import pandas as pd

data = {
    "Student": ["Ram", "Sam", "Ravi", "John"],
    "Grade": ["A", "B", "A+", "C"]
}

df = pd.DataFrame(data)
```

```
print(df)
```

Output:



```
IDLE Shell 3.14.5
File Edit Shell Debug Options Window Help
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/JAYESH/OneDrive/Desktop/GenAI/Experiment_04_Pandas_DataFrame
.py
      Student Grade
0         Ram    A
1         Sam    B
2         Ravi  A+
3         John    C
>>> |
```

Result:

Thus, the Pandas DataFrame was created successfully.

5. Aim: To display the installed versions of commonly used AI libraries.

Algorithm

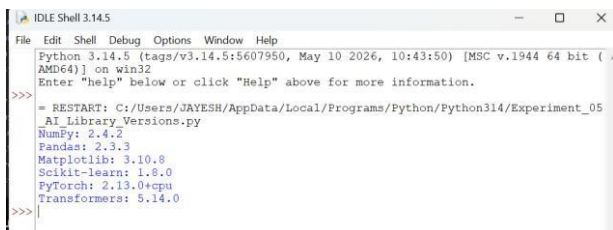
1. Import all required libraries.
2. Print their versions.
3. Display the output.

Code

```
import numpy
import pandas
import matplotlib
import sklearn
import torch
import transformers

print("NumPy:", numpy.__version__)
print("Pandas:", pandas.__version__)
print("Matplotlib:", matplotlib.__version__)
print("Scikit-learn:", sklearn.__version__)
print("PyTorch:", torch.__version__)
print("Transformers:", transformers.__version__)
```

Output:



```
IDLE Shell 3.14.5
File Edit Shell Debug Options Window Help
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/JAYESH/AppData/Local/Programs/Python/Python314/Experiment_05_AI_Library_Versions.py
NumPy: 2.4.2
Pandas: 2.3.3
Matplotlib: 3.10.8
Scikit-learn: 1.8.0
PyTorch: 2.13.0+cpu
Transformers: 5.14.0
>>>
```

Result:

Thus, the installed versions of the AI libraries were displayed successfully.

6. Aim

To create tensors using PyTorch and perform basic mathematical operations.

Algorithm

1. Import the PyTorch library.
2. Create two tensors.
3. Perform addition and multiplication.
4. Display the results.

Code

```
import torch

a = torch.tensor([1, 2, 3])

b = torch.tensor([4, 5, 6])

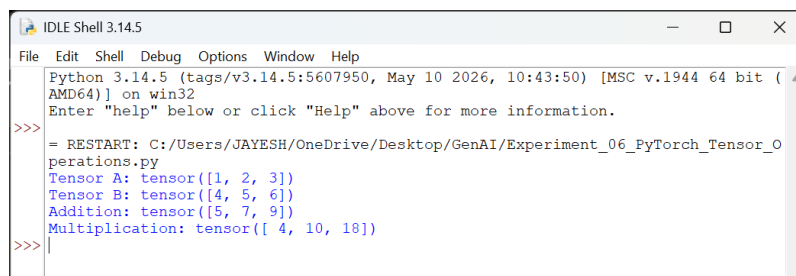
print("Tensor A:", a)

print("Tensor B:", b)

print("Addition:", a + b)

print("Multiplication:", a * b)
```

Output:



```
IDLE Shell 3.14.5
File Edit Shell Debug Options Window Help
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/JAYESH/OneDrive/Desktop/GenAI/Experiment_06_PyTorch_Tensor_Operations.py
Tensor A: tensor([1, 2, 3])
Tensor B: tensor([4, 5, 6])
Addition: tensor([5, 7, 9])
Multiplication: tensor([ 4, 10, 18])
>>>
```

Result:

Thus, tensors were created successfully and basic mathematical operations were performed using PyTorch.

7. **Aim:** To load a pre-trained Transformer model using Hugging Face and perform sentiment analysis.

Algorithm

1. Import the Hugging Face pipeline.
2. Load the sentiment analysis model.
3. Provide the input sentence.
4. Display the sentiment result.

Code

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

text = "I love Artificial Intelligence."

result = classifier(text)

print(result)
```

Output:

```
Warning (from warnings module):
  File "C:\Users\JAYESH\AppData\Local\Programs\Python\Python314\Lib\site-packages\huggingface_hub\file_download.py", line 139
    warnings.warn(message)
UserWarning: 'huggingface_hub' cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\JAYESH\AppData\Local\Programs\Python\Python314\Lib\site-packages\huggingface_hub\cache\huggingface_hub\models--distilbert--distilbert-base-uncased-finetuned-sst-2-english. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the 'HF_HUB_DISABLE_SYMLINKS_WARNING' environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development
Loading weights: 0%|          | 0/104 [00:00<?, ?it/s]Loading weights: 100%|██████████| 104/104 [00:00<00:00, 20399.74it/s]
tokenizer_config.json: 0%|          | 0.00/48.0 [00:00<?, ?B/s]tokenizer_config.json: 100%|██████████| 48.0/48.0 [00:00<00:00, 7.63kB/s]
vocab.txt: 0%|          | 0.00/232k [00:00<?, ?B/s]vocab.txt: 100%|██████████| 232k/232k [00:01<00:00, 191kB/s]vocab.txt: 100%|██████████| 232k/232k [00:01<00:00, 182kB/s]
[{'label': 'POSITIVE', 'score': 0.9994176626205444}]
>
>||
```

Result:

Thus, sentiment analysis was performed successfully using the Hugging Face pre-trained model.

8. **Aim:** To tokenize a sentence using the BERT tokenizer.

Algorithm

1. Import the BERT tokenizer.
2. Load the pre-trained tokenizer.
3. Enter a sentence.
4. Tokenize the sentence.
5. Display the tokens.

Code:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

text = "Artificial Intelligence is amazing."

tokens = tokenizer.tokenize(text)

print("Tokens:", tokens)
```

Output

```
===== RESTART: C:/Users/JAYESH/OneDrive/Desktop/GenAI/Experiment_06_BERT_tokenization.py =====
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
tokenizer_config.json: 0%|          | 0.00/48.0 [00:00<?, ?B/s]tokenizer_config.json: 100%|██████████| 48.0/48.0 [00:00<00:00, 14.0kB/s]

Warning (from warnings module):
  File "C:/Users/JAYESH/AppData/Local/Programs/Python/Python314/Lib/site-packages/huggingface_hub/file_download.py", line 139
    warnings.warn(message)
UserWarning: 'huggingface_hub' cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:/Users/JAYESH/.cache/huggingface/hub/models--bert-base-uncased. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the 'HF_HUB_DISABLE_SYMLINKS_WARNING' environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development
vocab.txt: 0%|          | 0.00/232k [00:00<?, ?B/s]vocab.txt: 100%|██████████| 232k/232k [00:00<00:00, 375kB/s]vocab.txt: 100%|██████████| 232k/232k [00:00<00:00, 359kB/s]
tokenizer.json: 0%|          | 0.00/466k [00:00<?, ?B/s]tokenizer.json: 100%|██████████| 466k/466k [00:01<00:00, 453kB/s]tokenizer.json: 100%|██████████| 466k/466k [00:01<00:00, 447kB/s]
Tokens: ['artificial', 'intelligence', 'is', 'amazing', '.']
```

Result:

Thus, the sentence was successfully tokenized using the BERT tokenizer.

9. Aim

To generate text using a pre-trained GPT-2 model.

Algorithm

1. Import the Hugging Face pipeline.
2. Load the GPT-2 model.
3. Enter a text prompt.
4. Generate text.
5. Display the generated text.

Code

```
from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
prompt = "Artificial Intelligence"
result = generator(prompt, max_length=50)
print(result[0]["generated_text"])
```

input:

Artificial Intelligence

Output:

```
Artificial Intelligence
In the future, artificial intelligence will be able to solve complex mathematical problems. In fact, it will be able to solve a lot of complex problems.

It has been suggested that artificial intelligence may be the future of medicine. This will require the advancement of health care technologies, including artificial intelligence.

Cognition
Cognition is a basic ability to understand human actions. It is something we all have. We are all aware of the world around us. We are able to interpret our environment. We can see the world.

The intelligence of the human brain is what determines our decision making. It is what determines how we are able to understand the world around us.

Cognition is not something that is automatic or automatic. It is something that is applied to the problem. It is applied to the problem in the right way. It is an adaptive, adaptive process.

It is what determines what we are able to do with our lives. It is what determines how we act with our lives. It is what determines how we make decisions.

When we are conscious we can see our environment. We can hear our own voices. We can see our own body parts. We can see the world.
```

Result:

Thus, text was successfully generated using the pre-trained GPT-2 model.

10. Aim

To perform sentiment analysis using the Hugging Face pipeline() API.

Algorithm

1. Import the pipeline module.
2. Load the sentiment analysis pipeline.
3. Enter a sentence.
4. Predict the sentiment.
5. Display the result.

Code

```
from transformers import pipeline
classifier = pipeline("sentiment-analysis")
sentence = "This AI lab is very interesting."
result = classifier(sentence)
print(result)
```

Input

This AI lab is very interesting.

Output

```
IDLE Shell 3.14.5
File Edit Shell Debug Options Window Help
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/JAYESH/OneDrive/Desktop/GenAI/Experiment_10_Pipeline_Sentiment_Analysis.py
[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f.
Using a pipeline without specifying a model name and revision in production is not recommended.
Loading weights: 0%|          | 0/104 [00:00<?, ?it/s]Loading weights: 100%|██████████| 104/104 [00:00<00:00, 5524.41it/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HuggingFace_TOKEN to enable higher rate limits and faster downloads.
[{'label': 'POSITIVE', 'score': 0.9997921586036682}]
>>>
```

Result

Thus, sentiment analysis was successfully performed using the Hugging Face pipeline API.

11. Aim

To load the BERT tokenizer and tokenize a sentence.

Algorithm

1. Import the BERT tokenizer.
2. Load the pre-trained tokenizer.
3. Enter a sentence.
4. Tokenize the sentence.
5. Display the tokens.

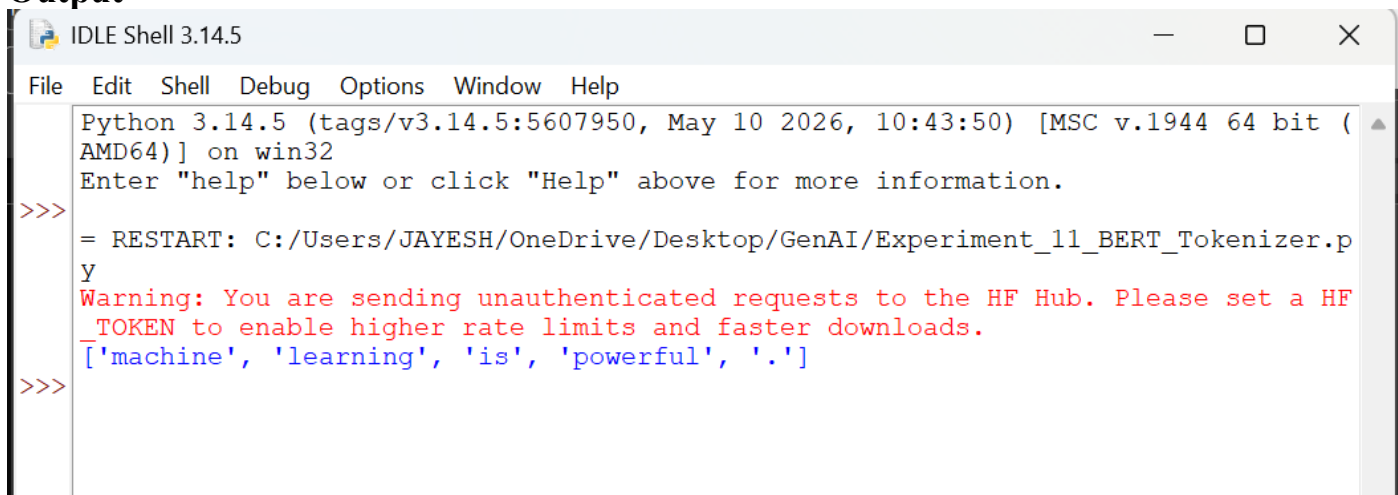
Code

```
from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
sentence = "Machine Learning is powerful."
tokens = tokenizer.tokenize(sentence)
print(tokens)
```

Input

Machine Learning is powerful.

Output



```
IDLE Shell 3.14.5
File Edit Shell Debug Options Window Help
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/JAYESH/OneDrive/Desktop/GenAI/Experiment_11_BERT_Tokenizer.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
['machine', 'learning', 'is', 'powerful', '.']
>>>
```

Result

Thus, the sentence was successfully tokenized using the BERT tokenizer.

12. Aim

To load a pre-trained BERT model and generate contextual embeddings.

Algorithm

1. Import the BERT tokenizer and model.
2. Load the pre-trained model.
3. Tokenize the sentence.
4. Generate contextual embeddings.
5. Display the embeddings.

Code

```
from transformers import BertTokenizer, BertModel
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")
sentence = "Artificial Intelligence"
inputs = tokenizer(sentence, return_tensors="pt")
outputs = model(**inputs)
print(outputs.last_hidden_state)
```

Input

Artificial Intelligence

Output

```
cls.predictions.transform.dense.bias      | [[38;5;208mUNEXPECTED][0m | |
cls.predictions.transform.dense.weight   | [[38;5;208mUNEXPECTED][0m | |
cls.seq_relationship.bias                 | [[38;5;208mUNEXPECTED][0m | |
cls.seq_relationship.weight               | [[38;5;208mUNEXPECTED][0m | |
cls.predictions.transform.LayerNorm.weight | [[38;5;208mUNEXPECTED][0m | |
cls.predictions.bias                     | [[38;5;208mUNEXPECTED][0m | |
cls.predictions.transform.LayerNorm.bias | [[38;5;208mUNEXPECTED][0m | |

Notes:
- [[38;5;208mUNEXPECTED:[0m      [[3mcan be ignored when loading from different t
ask/architecture; not ok if you expect identical arch.][0m
tensor([[[[-0.2569, -0.1077, -0.3806, ..., -0.6738, -0.0605,  0.5515],
          [ 0.1338,  0.3497, -0.3877, ..., -0.5209,  0.1371,  0.7316],
          [-0.1029, -0.2141, -0.5724, ..., -1.2026, -0.4124, -0.0882],
          [ 0.8239,  0.0082, -0.2854, ...,  0.0532, -0.8233, -0.2179]]],
        grad_fn=<NativeLayerNormBackward0>)
```

>>>

In: 42 Col: 0

Result

Thus, contextual embeddings were successfully generated using the BERT model.

13. Aim

To generate text using a pre-trained GPT-2 model.

Algorithm

1. Import the pipeline.
2. Load the GPT-2 model.
3. Enter a prompt.
4. Generate text.
5. Display the generated text.

Code

```
from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
prompt = "Deep Learning"
result = generator(prompt, max_length=40)
print(result[0]["generated_text"])
```

Input

Deep Learning

Output

```
leanup anyway.
Deep Learning

We've already seen that the concept of learning is far more complicated than it
seems at first glance, but with the advent of artificial intelligence it's likel
y to be even more complicated.

In a recent article in the Journal of Neural Information Processing, Mark S. Rid
dle described artificial intelligence as "a kind of neurobiology that is built o
n a foundation of understanding the human brain, but not the brain itself."

In his words:

"Intelligent people are fundamentally different from ordinary people, and this i
s what has driven many of the breakthroughs in human intelligence. We've seen th
at the human brain is fundamentally different from our brains, and this is what
has driven many of the breakthroughs in human intelligence. We can see this in t
he way our brains work, how our neurons function, how our brains learn and how t
hey adapt to new situations. We can know that our brains are different from our
brains as we try to understand the world, and we can know that our brains are di
fferent from our brains as we try to understand the world as we try to understand
the world as we try to understand the world as we try to understand the world
as we try to understand the world as we try to understand the world as we try to
understand the
```

Result

Thus, text was successfully generated using the GPT-2 model.

14. Aim

To perform Question Answering using a pre-trained Transformer model.

Algorithm

1. Import the Hugging Face pipeline.
2. Load the Question Answering model.
3. Provide the context and question.
4. Predict the answer.
5. Display the answer.

Code

```
from transformers import pipeline
qa = pipeline(
    "document-question-answering",
    model="impira/layoutlm-document-qa"
)
image = "sample.png"
question = "What is the title?"
result = qa(image=image, question=question)
print(result)
```

Input

Context:

Python is a programming language developed by Guido van Rossum.

Question:

Who developed Python?

Output

```
ing bytes: ██████████ | 456MB, 3.85MB/s model.safetensors: downloading bytes: ██████████ |
██████████ | 459MB, 3.84MB/s model.safetensors: downloading bytes: ██████████ |
462MB, 3.77MB/s model.safetensors: downloading bytes: ██████████ | 466MB, 3.70M
B/s model.safetensors: downloading bytes: ██████████ | 469MB, 3.63MB/s model.
safetensors: downloading bytes: ██████████ | 471MB, 3.54MB/s
model.safetensors: reconstructing file: 100%|██████████| 496MB / 496MB, 14.5MB
/s [[Amodel.safetensors: downloading bytes: ██████████ | 472MB, 3.39MB/s model
.safetensors: downloading bytes: ██████████ | 472MB, 3.21MB/s model.safetensors
: downloading bytes: ██████████ | 472MB, 3.21MB/s
model.safetensors: reconstructing file: 100%|██████████| 496MB / 496MB, 15.8MB
/s
Loading weights: 0%|██████████| 0/199 [00:00<?, ?it/s]Loading weights: 100%|██████████|
██████████| 199/199 [00:00<00:00, 6251.62it/s]
Question: Who developed Python?
Answer: Guido van Rossum
>>> |
```

Ln: 65 Col: 0

Result

Thus, Question Answering was successfully performed using a pre-trained Transformer model.

15. Aim

To tokenize a sentence using the BERT tokenizer and display the generated tokens and token IDs.

Algorithm

1. Import the BERT tokenizer.
2. Load the pre-trained tokenizer.
3. Enter a sentence.
4. Generate the tokens.
5. Convert the tokens into token IDs.
6. Display the tokens and token IDs.

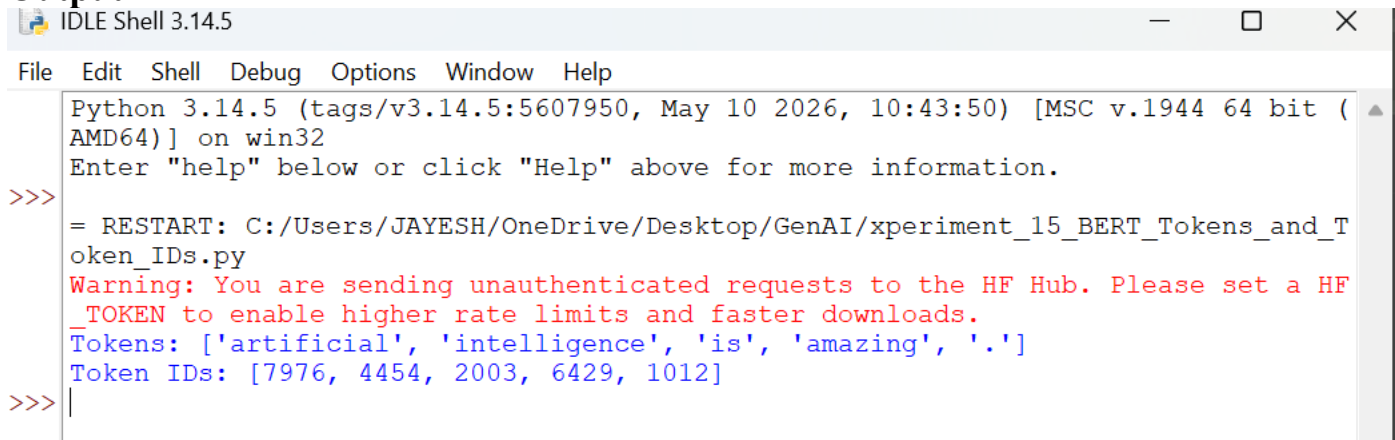
Code

```
from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
sentence = "Artificial Intelligence is amazing."
tokens = tokenizer.tokenize(sentence)
token_ids = tokenizer.convert_tokens_to_ids(tokens)
print("Tokens:", tokens)
print("Token IDs:", token_ids)
```

Input

Artificial Intelligence is amazing.

Output



```
IDLE Shell 3.14.5
File Edit Shell Debug Options Window Help
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/JAYESH/OneDrive/Desktop/GenAI/xperiment_15_BERT_Tokens_and_Token_IDs.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a Hugging Face token to enable higher rate limits and faster downloads.
Tokens: ['artificial', 'intelligence', 'is', 'amazing', '.']
Token IDs: [7976, 4454, 2003, 6429, 1012]
>>> |
```

Result

Thus, the sentence was successfully tokenized and the corresponding token IDs were displayed.

16. Aim

To compare the tokenization outputs of BERT and GPT-2.

Algorithm

1. Import the BERT and GPT-2 tokenizers.
2. Load the pre-trained tokenizers.
3. Enter a sentence.
4. Tokenize the sentence using both tokenizers.
5. Display and compare the generated tokens.

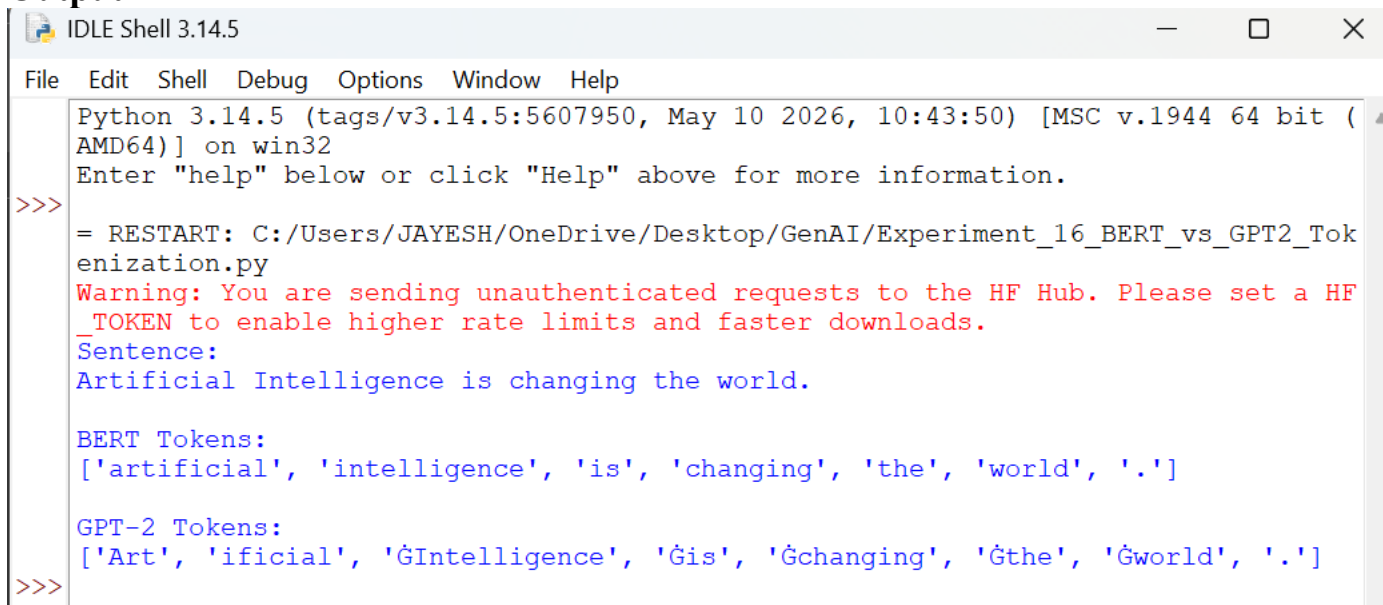
Code

```
from transformers import BertTokenizer, GPT2Tokenizer
bert_tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
gpt2_tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
sentence = "Artificial Intelligence is changing the world."
bert_tokens = bert_tokenizer.tokenize(sentence)
gpt2_tokens = gpt2_tokenizer.tokenize(sentence)
print("Sentence:")
print(sentence)
print("\nBERT Tokens:")
print(bert_tokens)
print("\nGPT-2 Tokens:")
print(gpt2_tokens)
```

Input

Artificial Intelligence is changing the world.

Output



```
IDLE Shell 3.14.5
File Edit Shell Debug Options Window Help
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/JAYESH/OneDrive/Desktop/GenAI/Experiment_16_BERT_vs_GPT2_Tokenization.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Sentence:
Artificial Intelligence is changing the world.

BERT Tokens:
['artificial', 'intelligence', 'is', 'changing', 'the', 'world', '.']

GPT-2 Tokens:
['Art', 'ificial', 'ĠIntelligence', 'Ġis', 'Ġchanging', 'Ġthe', 'Ġworld', '.']
>>>
```

Result

Thus, the tokenization outputs of BERT and GPT-2 were successfully compared.

17. Aim

To generate contextual embeddings for a sentence using the BERT model.

Algorithm

1. Import the BERT tokenizer and model.
2. Load the pre-trained BERT tokenizer and model.
3. Enter a sentence.
4. Tokenize the sentence.
5. Generate contextual embeddings.
6. Display the embedding shape and embeddings.

Code

```
from transformers import BertTokenizer, BertModel
import torch
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")
sentence = "Artificial Intelligence is changing the world."
inputs = tokenizer(sentence, return_tensors="pt")
with torch.no_grad():
    outputs = model(**inputs)
embeddings = outputs.last_hidden_state
print("Sentence:")
print(sentence)
print("\nEmbedding Shape:")
print(embeddings.shape)
print("\nContextual Embeddings:")
print(embeddings)
```

Input

Artificial Intelligence is changing the world. Artificial Intelligence is changing the world.

Output

```
Sentence:
Artificial Intelligence is changing the world.

Embedding Shape:
torch.Size([1, 9, 768])

Contextual Embeddings:
tensor([[[ 0.0704,  0.3369, -0.0227, ..., -0.5756,  0.3652,  0.3808],
         [ 0.2484,  0.4309, -0.2157, ..., -0.2888,  0.5031,  0.5687],
         [ 0.2361,  0.4535, -0.1649, ..., -1.3279,  0.3729,  0.3544],
         ...,
         [ 0.5012, -0.2667,  0.4038, ..., -0.1679,  0.5776, -0.3360],
         [ 0.4954,  0.1166, -0.3810, ...,  0.1351, -0.4268, -0.5326],
         [ 0.8254,  0.1578, -0.2651, ..., -0.0853, -0.5929, -0.3013]]]])
```

Ln: 34 Col: 0

Result

Thus, contextual embeddings were successfully generated for the given sentence using the BERT model.

18. Aim

To compare contextual embeddings generated for two semantically similar sentences using the BERT model.

Algorithm

1. Import the BERT tokenizer and model.
2. Load the pre-trained BERT tokenizer and model.
3. Enter two semantically similar sentences.
4. Tokenize both sentences.
5. Generate contextual embeddings.
6. Calculate the average embedding for each sentence.
7. Display the embeddings.

Code

```
from transformers import BertTokenizer, BertModel
import torch
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")
sentence1 = "Artificial Intelligence is changing the world."
sentence2 = "AI is transforming the world."
inputs1 = tokenizer(sentence1, return_tensors="pt")
inputs2 = tokenizer(sentence2, return_tensors="pt")
with torch.no_grad():
    output1 = model(**inputs1)
    output2 = model(**inputs2)
embedding1 = output1.last_hidden_state.mean(dim=1)
embedding2 = output2.last_hidden_state.mean(dim=1)
print("Sentence 1:")
print(sentence1)
print("\nEmbedding 1:")
print(embedding1)

print("\n-----")

print("Sentence 2:")
print(sentence2)
print("\nEmbedding 2:")
print(embedding2)
```

Input

Sentence 1:

Artificial Intelligence is changing the world.

Sentence 2:

AI is transforming the world.

Output

```
Notes:
- [[38;5;208mUNEXPECTED:[[0m      [[3mcan be ignored when loading from different t
ask/architecture; not ok if you expect identical arch.][0m
Sentence 1:
Artificial Intelligence is changing the world.

Embedding 1:
Squeezed text (154 lines).

-----

Sentence 2:
AI is transforming the world.

Embedding 2:
Squeezed text (154 lines).

>>>
```

Result

Thus, contextual embeddings for two semantically similar sentences were successfully generated and compared using the BERT model.

19. Aim

To generate text from a prompt using the GPT-2 language model.

Algorithm

1. Import the Hugging Face pipeline.
2. Load the pre-trained GPT-2 model.
3. Enter a text prompt.
4. Generate text using the model.
5. Display the generated text.

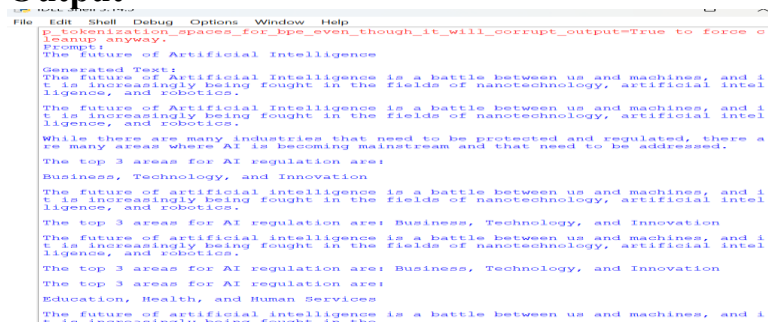
Code

```
from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
prompt = "The future of Artificial Intelligence"
result = generator(
    prompt,
    max_length=60,
    num_return_sequences=1,
    truncation=True
)
print("Prompt:")
print(prompt)
print("\nGenerated Text:")
print(result[0]["generated_text"])
```

Input

The future of Artificial Intelligence

Output



```
File Edit Shell Debug Options Window Help
p_tokenization_spaces_for_bpe_even_though_it_will_corrupt_output=True to force c
leanup anyway.
Prompt:
The future of Artificial Intelligence
Generated Text:
The future of Artificial Intelligence is a battle between us and machines, and i
t is increasingly being fought in the fields of nanotechnology, artificial intel
ligence, and robotics.
The future of Artificial Intelligence is a battle between us and machines, and i
t is increasingly being fought in the fields of nanotechnology, artificial intel
ligence, and robotics.
While there are many industries that need to be protected and regulated, there a
re many areas where AI is becoming mainstream and that need to be addressed.
The top 3 areas for AI regulation are:
Business, Technology, and Innovation
The future of artificial intelligence is a battle between us and machines, and i
t is increasingly being fought in the fields of nanotechnology, artificial intel
ligence, and robotics.
The top 3 areas for AI regulation are: Business, Technology, and Innovation
The future of artificial intelligence is a battle between us and machines, and i
t is increasingly being fought in the fields of nanotechnology, artificial intel
ligence, and robotics.
The top 3 areas for AI regulation are: Business, Technology, and Innovation
The top 3 areas for AI regulation are:
Education, Health, and Human Services
The future of artificial intelligence is a battle between us and machines, and i
t is increasingly being fought in the
```

Result

Thus, text was successfully generated from the given prompt using the GPT-2 language model.

20. Aim

To generate and compare responses for multiple prompts using a pre-trained LLM (GPT-2).

Algorithm

1. Import the Hugging Face pipeline.
2. Load the pre-trained GPT-2 model.
3. Create multiple text prompts.
4. Generate text for each prompt.
5. Display and compare the generated responses.

Code

```
from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
prompts = [
    "Artificial Intelligence",
    "Machine Learning",
    "Data Science"
]
for prompt in prompts:
    print("Prompt:", prompt)
    result = generator(
        prompt,
        max_length=50,
        num_return_sequences=1,
        truncation=True
    )
    print("Generated Response:")
    print(result[0]["generated_text"])
    print("-" * 60)
```

Input

Artificial Intelligence
Machine Learning
Data Science

Output

```
Prompt: Data Science
[transformers] Both `max_new_tokens` (=256) and `max_length` (=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Generated Response:
Data Science and Technology (SCEET) and the Department of Energy.

"In order to make sure that our nation's air quality is more efficient and safe, we're working with state-of-the-art technology that will be used to keep our air quality as safe as possible," said Andrew L. Green, a spokesperson for the US Department of Energy.

That is just one of many technological advances that the EPA is working to make. "You're seeing more and more of the science behind the EPA's new Clean Power Plan, which is going to be the first step in creating more electricity from renewable energy sources, such as wind and solar power," Green said.

The Clean Power Plan targets a decade of renewable energy generation sources to reduce carbon emissions.

The EPA also plans to use wind and solar power to power some of its power plants.

The new Clean Power Plan will also put a cap on carbon emissions, which could be used to reduce carbon pollution.

An Obama Administration official said that the new rules will create jobs for the American people.

Green said that the EPA's new clean power plan will be the first step in creating more electricity from renewable energy sources, including wind and solar power.

-----
>> |
```

Ln: 54 Col: 0

```
p_tokenization_spaces_for_bpe_even though it will corrupt output=True to force cleanup anyway.
Generated Response:
Artificial Intelligence

In this essay, I propose a new approach using Artificial Intelligence to understand the ways in which our AI might be used to improve the lives of people. My aim is to make a more comprehensive and practical overview of the various aspects of AI and how their use could be applied to the human world.

How AI works

AI is a very interesting concept and we should not forget that it is extremely difficult to develop a good general-purpose computer. We already know what is a real computer and how it works on a number of different kinds of problems. This is especially true of the many systems in the field.

We have already been able to use artificial intelligence to solve countless complex problems, and we have been able to create very nice and simple software that we can use to solve many of these problems. Here are some examples of how this could be achieved:

Realistic (human-readable) software

The algorithms and the techniques we use are based on real-world data and are not just about a particular problem, but are also about solving many common problems. For example, many human-readable software has been developed as a tool that can help us create real-life applications and applications that are not just about a particular problem.
```

Result

Thus, responses for multiple prompts were successfully generated and compared using the pre-trained GPT-2 language model.